STATUS: APROVADO PARA IMPLANTAÇÃO

Plano de Implantação & Especificação Técnica

Sistema de Monitoramento de Rede Distribuído (Throughput & Latência)

Projeto: Network Probe & Central Collector
Versão: 1.0.0 (Produção Homologada)
Data: 11 de Setembro de 2026
Stack: Go 1.22+ • Wails v2 • MongoDB Time Series

1. RESUMO EXECUTIVO & ARQUITETURA GERAL

Este projeto estabelece uma infraestrutura corporativa robusta para monitoramento contínuo de conectividade, vazão (Download/Upload em Mbps) e latência de gateway (Ping em ms) entre data centers e filiais. A arquitetura é 100% desacoplada: os agentes desktop operam autonomamente com banco local e reportam métricas via HTTPS ao nó central em formato de séries temporais de alta granularidade.

Client Probe (Desktop Windows)

- **Framework:** Wails v2 + Go 1.22 (Dark Theme)
- **Ciclo:** Goroutine Ticker (1 hora) + Disparo Manual
- **Resiliência:** SQLite Local (`probe_local.db`)
- **Frontend:** Chart.js (Download, Upload e Ping)

POST HTTPS
Bearer Auth
➔➔➔
JSON Payload

Central Server (Linux Debian)

- **Backend:** Go 1.22 com Driver Oficial Mongo v2
- **Time Series:** `network_central.metrics`
- **Config:** `timeField: timestamp | granularity: hours`
- **Dashboard:** Web responsivo com filtros por nó

2. CONTRATO DA API DE TELEMETRIA (`POST /API/V1/METRICS`)

A comunicação entre cada sonda e o servidor central ocorre via HTTPS utilizando o cabeçalho `Authorization: Bearer <api_token>` e payload estruturado em JSON com precisão decimal:

```
{
  "node_id": "minipc-dc-01",          "location": "Guarapuava-DC1",
  "timestamp": "2026-09-11T15:00:00Z",
  "download_mbps": 850.5,       "upload_mbps": 720.0,       "ping_gateway": 1.2
}
```

3. DETALHAMENTO TÉCNICO DOS COMPONENTES

Client Desktop (`/client-probe`) Wails v2

- ✓ **Configuração (`config.json`):** Parâmetros de nó, servidor e token carregados na inicialização e editáveis em tempo de execução via UI.
- ✓ **Agendamento Automático:** Goroutine em segundo plano executando testes a cada 1 hora sem bloqueio da interface gráfica.
- ✓ **Medição Real de Rede:** Ping para o gateway padrão via ICMP nativo / TCP socket e cálculo de throughput Download/Upload em Mbps.
- ✓ **Persistência Local (SQLite):** Histórico gravado em `probe_local.db` para operação offline resiliente.
- ✓ **Interface Visual:** Dark slate com acentos em ciano e roxo, gráficos de 24h, status badge dinâmico e botão com spinner.

Central Server (`/central-server`) Debian / Mongo v2

- ✓ **Driver Oficial MongoDB v2:** Conexão via `go.mongodb.org/mongo-driver/v2/mongo` com pooling e retry automático.
- ✓ **MongoDB Time Series:** Coleção nativa `metrics` configurada com `timeField: "timestamp"` , `metaField: "metadata"` e `granularity: "hours"` .
- ✓ **API REST com Bearer Auth:** Middleware de validação de token e endpoint analítico `GET /api/v1/dashboard/metrics` .
- ✓ **Dashboard Web Integrado:** Servido em `GET /` com seletor de nós, cartões de resumo e comparativo entre data centers.
- ✓ **Produção Debian:** Docker Compose, Dockerfile multi-stage e service unit para systemd.

4. MATRIZ DE ESTRUTURA DE CÓDIGO IMPLEMENTADA

ARQUIVO / MÓDULO	STACK / DRIVER	ATRIBUIÇÃO E RESPONSABILIDADE TÉCNICA
`client-probe/main.go`	Go 1.22 / Wails v2	Ponto de entrada do executável Windows, binding de métodos e ciclo de vida da janela.
`client-probe/probe.go`	Go Net / Exec	Motor de teste: Ping do gateway (ICMP/TCP) e throughput de Download e Upload em Mbps.
`client-probe/storage.go`	SQLite / HTTP Client	Persistência local (`probe_local.db`), controle de sincronismo e envio HTTPS tolerante a falhas.
`client-probe/frontend/`	HTML5 / CSS / Chart.js	Interface gráfica Wails com gráficos de 24h, botão de execução manual e modal de configuração.
`central-server/main.go`	Go 1.22+ Standard Lib	Servidor HTTP com roteamento REST, entrega estática do frontend e graceful shutdown.
`central-server/db/mongodb.go`	Mongo Driver v2	Inicialização da coleção Time Series, criação de índices e consultas analíticas de 24 horas.
`central-server/handlers/`	Go Handlers & Middlewares	Middleware Bearer Auth, endpoint de ingestão (POST), telemetria (GET) e endpoints de speedtest.
`central-server/public/`	Web / Chart.js	Painel Web NOC responsivo com seletor de mini PCs, cartões de status e gráficos comparativos.
`central-server/docker-compose.yml`	Docker Compose	Orquestrador do serviço Go + MongoDB 7.0 Time Series pronto para implantação em Debian.
`central-server/systemd/`	Systemd Unit	Arquivo de serviço `network-monitor.service` para execução nativa em segundo plano no Linux.

5. ROTEIRO E FASES DE IMPLANTAÇÃO EM PRODUÇÃO

FASE	AMBIENTE	PROCEDIMENTO TÉCNICO	CRITÉRIO DE HOMOLOGAÇÃO
Fase 1	VM Debian (NOC)	Subir MongoDB 7 e Go Central Server via Docker Compose	Containers `healthy` ; endpoint `/health` respondendo HTTP 200.
Fase 2	MongoDB	Validar criação automática da coleção Time Series `metrics`	Comando `db.getCollectionInfos()` confirma `granularity: hours` .
Fase 3	Mini PCs (Windows)	Compilar `probe-app.exe` via Wails e distribuir com `config.json`	App inicializa em segundo plano e grava primeiro teste no SQLite local.
Fase 4	End-to-End	Executar disparo manual e validar ingestão no Dashboard Central	Métricas surgem em tempo real no painel web e tabela de telemetria.

6. CRITÉRIOS DE QUALIDADE E RESILIÊNCIA OPERACIONAL

Resiliência e Tolerância a Quedas

• Quedas de internet na ponta não travam a sonda desktop: medições ficam retidas no SQLite local como `synced=0` .
• Servidor Central opera em modo não-bloqueante com timeout de 5s no MongoDB e graceful shutdown em sinais SIGINT/SIGTERM.

Performance e Escalabilidade

• O modelo MongoDB Time Series compacta leituras por hora e data center, reduzindo em até 70% o uso de disco.
• Frontend do Client e do Servidor utilizam Chart.js otimizado para renderização de até centenas de pontos sem perda de fluidez.